

Pipeline: Input Sources → Detect Source → Parse & Extract → Normalize → Merge → Confidence & Provenance → Projection → Validation → JSON Output

1. Problem Statement

Recruitment workflows ingest candidate data from highly fragmented sources, ranging from unstructured resume PDFs to structured recruiter CSVs. This pipeline addresses this fragmentation by parsing, normalising, resolving conflicts, and merging these inputs deterministically into a single canonical Pydantic profile.

2. System Architecture

The system is built on a clean pipeline pattern using SOLID design principles:

- **Ingestion:** Evaluates input files and routes to dedicated parsers.
- **Parsing Layer:** Extracts text blocks via pdfplumber and rows via pandas.
- **Normalization Layer:** Applies E.164, YYYY-MM, ISO-3166-1, and skill synonyms.
- **Merger Layer:** Implements precedence rules and resolves data conflicts.
- **Scoring & Provenance:** Audits lineage and computes confidence levels.
- **Projection Layer:** Restructures schemas dynamically per client output needs.

3. Canonical Schema

Defined strictly using Pydantic models to guarantee contract integrity:

- **candidate_id:** Deterministic string identifier generated per profile.
- **location:** Structured details containing *city*, *region*, and *country*.
- **links:** Endpoint map storing *linkedin*, *github*, *portfolio*, and *other* lists.
- **history:** Structured experience and education items chronologically sorted.
- **audit:** Flat lists logging field *provenance* and *field_confidence* weights.

4. Normalization Strategy

Preprocesses individual profiles before merging to guarantee consistency. The core normalization rules are mapped below:

Field	Target Format	Method / Library
Phones	E.164 format	phonenumbers library
Dates	YYYY-MM format	python-dateutil parser
Country	ISO-3166 alpha-2	Token matching lookup
Skills	Canonical synonyms	Exact maps + rapidfuzz

5. Merge & Conflict Resolution

Consolidates profiles using a strict source precedence hierarchy:

- Resume PDF (Priority 2) → Recruiter CSV (Priority 1) → Unknown (0)**
- **Precedence Policy:** Single fields select values from the highest priority source.
 - **Array Fields:** Union and deduplicate emails, phones, and skills lists.
 - **Work History:** Combines experience/education timelines, groups duplicates (same company & start year), and sorts chronologically.

6. Explainable Confidence

Assigns explicit certainty weights indicating source reliability. Confidences are mapped as follows:

Extraction Method	Score
Resume PDF direct extraction	0.95
Recruiter CSV direct parsing	0.90
Regex extraction patterns	0.80
Heuristics and text alignment rules	0.60
Unknown sources / default values	0.30

7. Provenance

Preserves data lineage by capturing metadata for every field value:

- Tracks **field** name, **source** filename, and extraction **method**.
- Audit records are aggregated in a query-friendly flat list layout.

8. Runtime Projection Layer

Runtime Config: Supports field inclusion, field renaming, normalization rules, provenance/confidence toggles, and configurable missing-value behavior without code changes.

- **Aliasing & Filters:** Uses *rename_fields* mapping and *include_fields* lists.
- **Missing Fields:** Toggles output between 'null' and 'exclude' (removing empty keys).

9. Validation & Edge Cases

- **Pydantic Validation:** Validates projected schema contracts. Synthesizes errors into detailed trace messages.
- **Edge cases:** Gracefully handles empty source files, missing credentials, duplicate candidates, and malformed files.

10. Assumptions & Future Improvements

- **Assumptions:** Raw text is extractable from PDFs. Candidates sharing primary emails represent the same entity.
- **Future Enhancements:** Support additional parsers (LinkedIn, GitHub JSON, ATS XML). Integrate OCR (Tesseract) for scanned resumes.